

Week 11

Formal Verification Methods

Jesus Sanchez

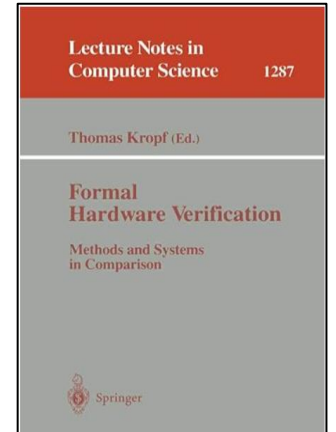
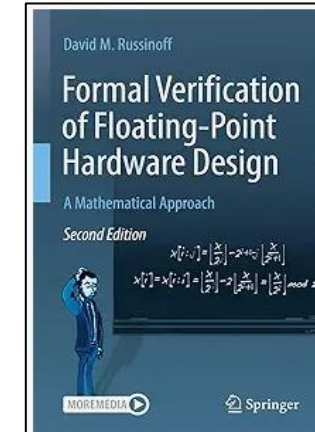
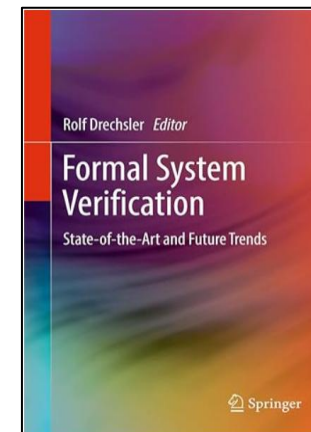
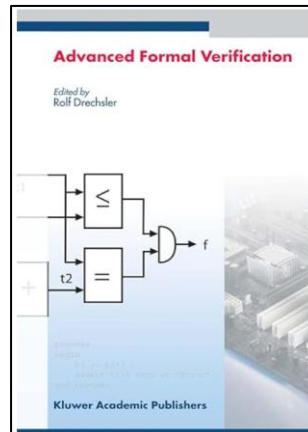
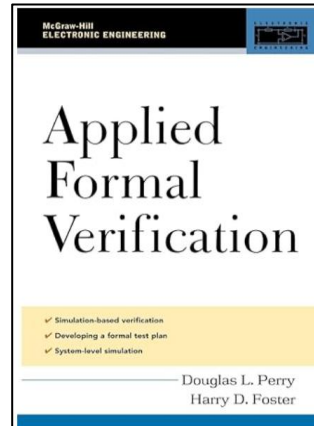
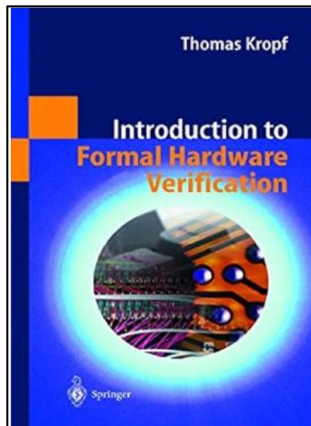
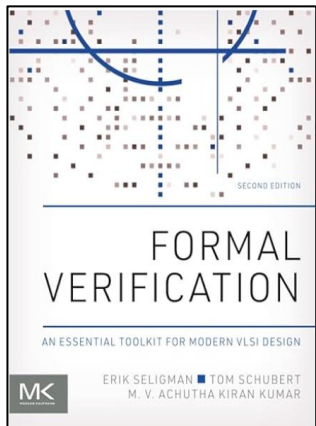
Dept. d'Arquitectura de Computadors
Universitat Politècnica de Catalunya

Processor Verification

Master in Innovation and Research in Informatics

Objectives of the Week

- Understand the **motivation** for formal verification
- Differentiate between **LEC** and **FPV**
- Apply **assertion**-based verification concepts
- Explore the role of **assumptions**
- Develop strategies for managing state space explosion



Agenda

1. Intro and motivation
2. Logical Equivalence Checking (LEC)
3. Formal Property Verification (FPV)
4. Adding Assumptions
5. Practical Formal Verification
6. Wrap-up and Self-Learning

Addressing the Verification Gap

- Chips are huge, chips are complex, chips are expensive
 - Bugs are even more expensive
- Simulation's Problem:
 - Pros: realistic scenarios, flexible
 - Cons: only explores what is stimulated/simulated
 - The state space for any meaningful ASIC design is nearly infinite
- The Corner Case:
 - The bugs that escape to production are usually control logic bugs in rare, hard-to-reach scenarios (e.g., arbitration during reset while a FIFO is almost full)

What is Formal Verification

- Use **mathematics + logic** to prove properties of a design
 - No stimulus needed → tool explores many/all possible input sequences
- **Formal's strength:**
 - Mathematically forces the tool to find these corner cases
 - It explores the **entire search space** defined by your assumptions
- **Core idea:**
 - Define **what must always be true**, tool proves or finds counter examples
- **Where is used at industry?**
 - Control-heavy blocks: protocols, arbiters, FIFOs
 - Critical logic: safety, security, reset sequences
 - Equivalence checking: RTL ↔ RTL, RTL ↔ Gate-level
 - Early bug-finding before full testbench exists

Simulation cannot exhaustively explore all states
Formal complements simulation

Types of Formal Verification

■ Two Main Branches of Formal Verification

1. Logical Equivalence Checking (LEC):

- **Goal:** Does the optimized/synthesized netlist have the exact same logic function as my original RTL code?
- **Use Case:** Standard part of every ASIC flow. Automated tool use

2. Formal Property Verification (FPV) :

- **Goal:** Does my RTL code function correctly according to my specification (SVA)?
- **Use Case:** Focus of this lecture and your lab. Requires manual property writing

Agenda

1. Intro and motivation
2. Logical Equivalence Checking (LEC)
3. Formal Property Verification (FPV)
4. Adding Assumptions
5. Practical Formal Verification
6. Wrap-up and Self-Learning

Logical Equivalence Checking (LEC)

- Mathematically prove or disprove two designs are equivalent
 - A mandatory step in the ASIC flow
 - Ensures correctness after transformations (synthesis, optimization, DFT insertion)
 - Detects unintended functional changes early → avoids costly silicon re-spins
- Example:
 - Input 1: Original RTL (SystemVerilog/VHDL code)
 - Input 2: Modified Netlist (e.g., after synthesis, optimization, or scan insertion)
 - Output: "Equivalent" or "Not Equivalent" + failure report

Essential for sign-off before tape-out

LEC in the Design Flow

- When is it used?

1. Post-Synthesis (RTL vs. Generic Gates)
2. Post-Optimization (Generic Gates vs. Vendor Library Gates)
3. Post-Physical Design (Gates vs. Placed & Routed cells)
4. Post-Scan Insertion (Ensure test logic didn't break functional logic)

LEC ensures that physical design engineers haven't inadvertently changed the functionality of your logic

Limitations of LEC

- **The Big Limitation:**

- LEC only checks for functional equivalence between two representations
- It does not validate correctness against the design specification

- **Key Points:**

- If the original RTL is wrong, LEC will confirm the netlist is equally wrong
- LEC cannot detect missing features, incorrect functionality, or spec violations
- It assumes the reference design (RTL) is the “golden” model

- **What LEC Does NOT Do:**

- Does not check timing, power, or physical constraints
- Does not verify design intent or compliance with requirements

Agenda

1. Intro and motivation
2. Logical Equivalence Checking (LEC)
3. Formal Property Verification (FPV)
4. Adding Assumptions
5. Practical Formal Verification
6. Wrap-up and Self-Learning Corner

Formal Property Verification (FPV)

- Mathematically prove or disprove design correctness
 - Exhaustively checking *all* possible scenarios
- Involves embedding behavioral "*properties*" within the design
 - Commonly in SVA (SystemVerilog Assertions)
 - Though PSL (Property Specification Language) is also used
- Then, when using the formal verification tool
 - The tool creates a mathematical model of your design RTL
 - It uses algorithms to systematically explore every single reachable state

Also called Assertion-Based Verification (ABV)

Dynamic Simulation vs FPV

Feature	Dynamic Simulation	Formal Property Verification
Method	Test Vectors, Stimulus	Mathematical Analysis
Coverage	Limited Scenarios	Exhaustive (All possibilities)
Goal	Find Bugs	Prove absence of bugs
Input	Testbench, Sequences	SVA Assertions/Assumptions
Output	Pass/Fail logs	Counterexamples (CEX) or Proofs
Best For	Datapath, High Throughput	Control Logic, Arbiters, FSMs

FPV Methodology Flow

1. Define the Scope

- Identify the critical control logic block (e.g., an arbiter, a FIFO pointer)

2. Write Properties (SVA assert)

- Define what must be true

How the design interacts
with the outside world
during property checking

3. Constrain the **Environment** (SVA assume)

- Tell the tool how to external world behaves ← Crucial Step!

4. Run the Tool

- Execute de formal analysis (using Questa/Jasper/VC-Formal)

5. Check the outcome



Shifting the Verification Mindset

- **From:** "I wrote a test to find this bug."
- **To:** "I wrote a property asserting this condition can never be violated."

- **From:** "Did my test run long enough?"
- **To:** "Is my property mathematically proved correct, or did the tool find a CEX?"

FPV is about defining correctness exhaustively,
not just testing behavior dynamically

Safety and Liveness Properties

■ Safety Properties (Something bad never happens):

- Examples:
 - FIFO overflow never occurs
 - Request and grant never high simultaneously
- These are easier for formal tools to prove

Assertions you already know
= safety properties

■ Liveness Properties (Something good eventually happens):

- Examples:
 - Every request is eventually granted
 - FSM never gets stuck forever
- These require careful assumptions about fairness (e.g., assuming the environment doesn't hold the input valid forever but turns it off when needed)

Formal checks **properties**, not testcases

How to Represent Properties?

■ Historically

- Proprietary and academic languages (Mid-1990s to early 2000s)
 - OVA (OpenVera Assertions, by Synopsys)
 - e-language (for Specman, by Verisity, later acquired by Cadence)
 - ForSpec (by Intel)
- OVL (Open Verification Library, by Accellera) (~2000)
- PSL (Property Specification Language, based on IBM's Sugar) (~2002-05)

■ Today:

- SVA (SystemVerilog Assertions) (~2002-05)

With SVA we can use same language for simulation and formal

Example 1: A Simple Arbiter

■ Specification

- We have 2 requesters (req0, req1) and 2 grants (gnt0, gnt1)
- Only one grant can be active at a time (gnt0 XOR gnt1)

■ Verilog snippet

```
module arbiter (  
    input  logic clk, reset_n,  
    input  logic req0, req1,  
    output logic gnt0, gnt1  
);  
    // ... design Logic goes here ...  
endmodule
```

FPV Property 1: Mutual Exclusion (Safety)

■ Goal

- Prove that the arbiter never grants access to both requesters simultaneously
- This is a **Safety** property ("something bad never happens")

■ SVA Code

```
property mutual_exclusion_p;  
    @(posedge clk) disable iff (!reset_n) !(gnt0 && gnt1);  
endproperty  
assert property (mutual_exclusion_p)  
    else $fatal(1, "Mutual exclusion violated!");
```

The formal tool tries everything to violate this assertion.
If it runs indefinitely without finding a CEX, it is proved.

FPV Property 2: Eventually Grants (Liveness)

■ Goal

- Prove that if a request is held high, a grant is eventually issued
- This is a **Liveness** property ("something eventual never happens")

■ SVA Code

```
property eventually_grants_p;  
    @(posedge clk) disable iff (!reset_n) req0 |-> ##[1:10] gnt0;  
endproperty  
assert property (eventually_grants_p)  
    else $fatal(1, "Eventually grants violated!");
```

Liveness proofs are complex.
You might hit "Inconclusive" proofs if your assertions aren't strong enough

10:00

Agenda

1. Intro and motivation
2. Logical Equivalence Checking (LEC)
3. Formal Property Verification (FPV)
4. Adding Assumptions
5. Practical Formal Verification
6. Wrap-up and Self-Learning

The Problem Without Restrictions

- If we run the tool with only the assert on the previous slides, the tool explores every illegal input combination as well
 - The tool might flag a failure during reset or if inputs violate system protocol



- We must tell the tool how our environment behaves using assume property
- Key Point:
 - **Assume** constrains the inputs
 - **Assert** checks the outputs

The Role of Assumptions

- Formal tools are "dumb" initially

- They assume any input value is possible at any time



- If your design is only meant to work when reset_n is high and stable inputs, you must tell the tool that.



- **assume property(...)** is how you constrain the environment.

- **Warning:**

- Too few assumptions = FPV finds irrelevant CEXs
 - Too many assumptions = FPV proves properties that won't hold in the real chip

Adding Assumptions

■ Goal

- We need to assume the reset signal works correctly
 - And that inputs are stable during a request cycle.

■ SVA Code:

Assume that reset is asserted for at least 3 cycles after startup

```
property assume_reset_works_p;  
    @(posedge clk) !reset_n |-> ##3 !reset_n;  
endproperty
```

Assertions: DUT guarantess

Assumptions: describe legal environment inputs

Example 2: FIFO Pointer Logic

■ Specification

- We want to verify that the full and empty flags are mutually exclusive.
- The most common bug: full and empty are true at the same time (usually a pointer wrap bug).

■ Verilog snippet

```
module fifo_ctrl (  
    input  logic clk, reset_n,  
    input  logic write_en, read_en,  
    output logic full, empty  
);  
    // ... design logic goes here ...  
endmodule
```

FPV for the FIFO Flags (Safety)

■ Goal

- This bug is nearly impossible to find with UVM unless you hit the exact pointer wrap-around scenario.
- FPV proves it impossible, instantly.

■ SVA Code

```
property full_and_empty_exclusive_p;  
    @(posedge clk) disable iff (!reset_n) !(full && empty);  
endproperty  
assert property (full_and_empty_exclusive_p)  
    else $fatal(1, "Full/empty exclusion violated!");
```

If your design is correct, the tool will instantly provide a "Proof Core" result.
If it's buggy, you get the CEX waveform that leads to this state

Constraining the FIFO Inputs

- Goal
 - We must make realistic assumptions about read_en and write_en.
- SVA Code

```
// Assume you can't read and write to a full FIFO at the same time  
assume property (@(posedge clk) disable iff (!reset_n)  
    !(full && read_en && write_en));  
  
// Assume you can't read from an empty FIFO  
assume property (@(posedge clk) disable iff (!reset_n)  
    !(empty && read_en));
```

Example 3: RTL Sketch (Simplified)

```
module handshake(  
    input  logic clk, rst_n, req,  
    output logic ack  
);  
    // ... design logic goes here ...  
endmodule
```

Safety/Liveness Properties

- **Safety:**
 - P1: ack never asserted without req
 - P2: No overlapping requests
- **Liveness:**
 - P3: After req, ack is asserted within 2 cycles

Writing Properties in SVA

- P1 Safety:

```
assert property (@(posedge clk)
    disable iff (!rst_n)
    ack |-> ($past(req) || $(past(req, 2))));
```

- P3 Liveness:

```
assert property (@(posedge clk)
    disable iff(!rst_n)
    req |-> ##[1:2] ack);
```


Environmental Assumptions

- P2: Req is single-cycle pulse

```
assume property (@(posedge clk)  
    disable iff(!rst_n)  
    req |-> !$past(req));
```

Agenda

1. Intro and motivation
2. Logical Equivalence Checking (LEC)
3. Formal Property Verification (FPV)
4. Adding Assumptions
5. Practical Formal Verification
6. Wrap-up and Self-Learning

What's the State Space?

- **Definition:**
 - “Set of all possible configuration or states that a system can be in, along with the transitions between those states.”
- **What is a state?**
 - Internal memory elements (aka. registers or flip-flops)
- **Do input/outputs of my design impact of the space size?**
 - No, but they determine the transitions between states
- **Example:**
 - Design with: 4x2-bit inputs, 2x1-bit outputs, and 8x6-bit registers
 - State space = $2^{8 \times 6} = 2^{48} = 281,474,976,710,656$

FPV for Combinational Designs?

- Yes, and it is a fundamental and often automated task
- We are essentially checking the relationship between inputs and outputs
- Example: 2:1 Multiplexer
 - Intent: “If the Select line (S) is 0, the Output (Y) must be equal to input A”
 - Assertion: **assert property** (S==0 | -> Y==A)
- Combinational formal verification is simpler
- Example: 4-bit Adder

```
// Check 4-bit Sum (S)  
property P_Sum_Correct;  
    (A + B + Cin)[3:0] == S;  
endproperty
```

```
// Check Carry-out (Cout)  
property P_Cout_Correct;  
    (A + B + Cin)[4] == Cout;  
endproperty
```

Environment Modeling

- Assume properties are used to reduce the state space
- Inputs must be constrained realistically:

- **Over-constraining**

- Misses real scenarios

Example:

Assuming a bus is always idle except during a transaction may hide bugs in idle handling

- **Under-constraining**

- False failures
 - Considers impossible scenarios
- Inconclusive proofs

Example:

Not constraining a FIFO's read/write enables may allow the tool to explore illegal simultaneous operations

Start with minimal, essential assumptions and incrementally add more as needed

Managing State Space Explosion

- The number of states grows exponentially with registers (2^N).
 - This is the #1 FPV limitation
- What's a reasonable size?
 - 2^{60} - 2^{80}
- Solutions:
 - **Abstraction:** Verify smaller blocks first, then the integrated top-level.
 - **Decomposition:** Break one massive property into smaller, easier-to-prove properties (aka. Divide-and-Conquer)
 - Example: a 3-stage pipelined design ($A \rightarrow B \rightarrow C$), each stage has 30 internal flip-flops
 - Task 1: Verify Stage A: prove A output is correct based on its inputs
 - Task 2: Verify Stage B: prove B output is correct on assumed correct A output
 - Task 3: Verify Stage C: prove C output is correct on assumed correct B output

FPV is Ideal for Control Logic

- FPV excels where UVM struggles
- Best Use Cases:
 - Arbiters (ensuring fairness and mutual exclusion)
 - Complex FSMs (proving all states are reachable/unreachable)
 - Protocol engines (AMBA AXI/APB/ACE connectivity checks)
 - Security/Trustzone verification
 - Configuration registers logic
- UVM is better for verifying the data moving through the pipes

Other Formal Apps

- Reset analysis
 - Registers properly initialized
- X-propagation
 - Uninitialized logic
- CDC checks
 - Clock domain crossing correctness
- Security/safety
 - Privilege, non-interference

Agenda

1. Intro and motivation
2. Logical Equivalence Checking (LEC)
3. Formal Property Verification (FPV)
4. Adding Assumptions
5. Practical Formal Verification
6. Wrap-up and Self-Learning

Summary of Key Concepts from this Session

- What is formal verification and how it does complement simulation
- LEC vs FPV
- Assertions vs assumptions
- When and how to use it in practice

Link to Next Week

- Week 12: Debug, Closure and Trend Strategies
 - Techniques to find, debug, and report bugs
 - How to close a verification task
 - Trends: Python, IA, etc.

Self-Learning Corner

- Articles:

- [A Gentle Introduction to Formal Verification](#)
- [Formal Verification vs Functional Verification: A Tale of Two Approaches](#)

- Slides:

- [Assertion-Based Verification](#), University of Bristol

- Videos:

- [Very Basic Introduction to Formal Verification](#)

“To verify is human.
To formally verify is divine.”

Hardware design proverb